VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

SHASHANK SHANTHARAM NAYAK (1BM23CS313)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February-May 2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by SHASHANK SHANTHARAM NAYAK (1BM23CS313), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2024-25. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Sarala D V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of Recursive and Non-recursive algorithms using asymptotic notations.
CO2	Apply various design techniques for the given problem.
CO3	Apply the knowledge of complexity classes P, NP, and NP-Complete and prove certain problems are NP-Complete
CO4	Design efficient algorithms and conduct practical experiments to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
//C program to implement topological sort using DFS
#include <stdio.h>
int n, a[10][10], res[10], s[10], top = 0;
void dfs(int, int, int[][10]);
void dfs_top(int, int[][10]);
int main()
{
	printf("Enter the no. of nodes:");
	scanf("%d", &n);
	int i, j;
	for (i = 0; i < n; i++) {
		for (j = 0; j < n; j++) {
			scanf("%d", &a[i][j]);
		}
	}
	dfs_top(n, a);
	printf("Solution: ");
	for (i = n - 1; i >= 0; i--) {
		printf("%d ", res[i]);
	}
	printf("\n");
	return 0;
}
void dfs_top(int n, int a[][10]) {
	int i;
```

```
        for (i = 0; i < n; i++) {
                s[i] = 0;
        }
        for (i = 0; i < n; i++) {
                if (s[i] == 0) {
                        dfs(i, n, a);
                }
        }
}
void dfs(int j, int n, int a[][10]) {
        s[j] = 1;
        int i;
        for (i = 0; i < n; i++) {
                if (a[j][i] == 1 && s[i] == 0) {
                        dfs(i, n, a);
                }
        }
        res[top++] = j;
}
```

Output:

```
> ./topo
Enter the no. of nodes:6
0 0 1 1 0 0
0 0 0 1 1 0
0 0 0 1 0 1
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Solution: 1 4 0 2 3 5
```

LeetCode Program related to Topological sorting

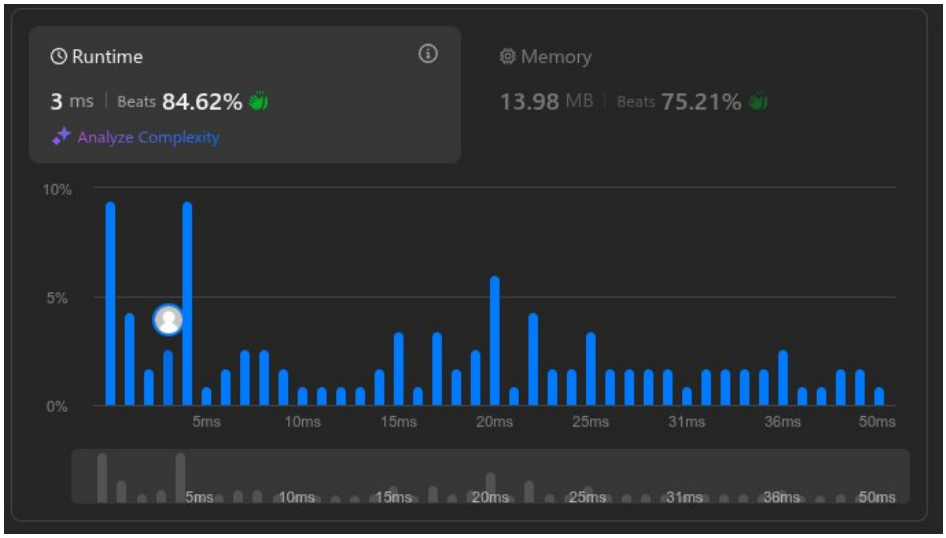
COURSE SCHEDULE II

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* findOrder(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize, int* returnSize) {
    // Step 1: Initialize adjacency list and in-degree array
    int* inDegree = (int*)calloc(numCourses, sizeof(int));
    int** adjList = (int**)malloc(numCourses * sizeof(int*));
    int* adjListSizes = (int*)calloc(numCourses, sizeof(int));

    // Initialize adjacency list
    for (int i = 0; i < numCourses; ++i) {
        adjList[i] = NULL;
    }

    // Fill adjacency list and in-degree array
    for (int i = 0; i < prerequisitesSize; ++i) {
        int course = prerequisites[i][0];
        int prerequisite = prerequisites[i][1];

        adjList[prerequisite] = (int*)realloc(adjList[prerequisite],
(adjListSizes[prerequisite] + 1) * sizeof(int));
        adjList[prerequisite][adjListSizes[prerequisite]++] = course;
        inDegree[course]++;
    }

    // Step 2: Initialize queue for courses with no prerequisites
    int* queue = (int*)malloc(numCourses * sizeof(int));
```

```
int front = 0, rear = 0;

for (int i = 0; i < numCourses; ++i) {
    if (inDegree[i] == 0) {
        queue[rear++] = i;
    }
}

// Step 3: Perform BFS and generate topological ordering
int* order = (int*)malloc(numCourses * sizeof(int));
int orderIndex = 0;

while (front < rear) {
    int course = queue[front++];
    order[orderIndex++] = course;

    for (int i = 0; i < adjListSizes[course]; ++i) {
        int nextCourse = adjList[course][i];
        if (--inDegree[nextCourse] == 0) {
            queue[rear++] = nextCourse;
        }
    }
}

// Step 4: Check if topological sort includes all courses
if (orderIndex != numCourses) {
    *returnSize = 0;
    free(order);
    order = NULL;
```

```
    } else {
        *returnSize = orderIndex;
    }

    // Free memory
    for (int i = 0; i < numCourses; ++i) {
        free(adjList[i]);
    }
    free(adjList);
    free(adjListSizes);
    free(inDegree);
    free(queue);

    return order;
}
```

Output:



Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include <stdio.h>
#include <stdlib.h>
void swap(int* a, int* b) {
	int temp = *a;
	*a = *b;
	*b = temp;
}
void generatePermutations(int arr[], int start, int end) {
	if (start == end) {
		for (int i = 0; i <= end; i++) {
			printf("%d ", arr[i]);
		}
		printf("\n");
	} else {
		for (int i = start; i <= end; i++) {
			swap(&arr[start], &arr[i]);
			generatePermutations(arr, start + 1, end);
			swap(&arr[start], &arr[i]); // backtrack
		}
	}
}
int main() {
	int n;
	printf("Enter the number of elements: ");
	scanf("%d", &n);
```

```
    int* arr = (int*)malloc(n * sizeof(int));
    printf("Enter the elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    generatePermutations(arr, 0, n - 1);
    free(arr);
    return 0;
}
```

Output:

```
> ./johnson
Enter the number of elements: 4
Enter the elements: 1 2 3 4
1 2 3 4
1 2 4 3
1 3 2 4
1 3 4 2
1 4 3 2
1 4 2 3
2 1 3 4
2 1 4 3
2 3 1 4
2 3 4 1
2 4 3 1
2 4 1 3
3 2 1 4
3 2 4 1
3 1 2 4
3 1 4 2
3 4 1 2
3 4 2 1
4 2 3 1
4 2 1 3
4 3 2 1
4 3 1 2
4 1 3 2
4 1 2 3
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void display(int* arr, int n) {
   for(int i = 0; i < n; i++) {
      printf("%d\t", arr[i]);
   }
   printf("\n");
}

void merge(int* arr, int low, int mid, int high) {
   int left_size = mid - low + 1;
   int right_size = high - mid;

   int left[left_size];
   int right[right_size];

   for (int i = 0; i < left_size; i++) {
      left[i] = arr[low + i];
   }
   for (int i = 0; i < right_size; i++) {
      right[i] = arr[mid + 1 + i];
   }
```

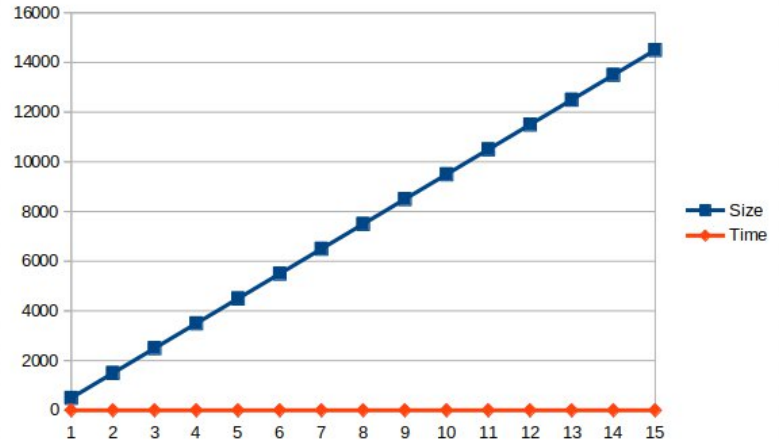
```
    int i = 0, j = 0, k = low;
    while (i < left_size && j < right_size) {
        if (left[i] <= right[j]) {
            arr[k] = left[i];
            i++;
        } else {
            arr[k] = right[j];
            j++;
        }
        k++;
    }

    while (i < left_size) {
        arr[k] = left[i];
        i++;
        k++;
    }

    while (j < right_size) {
        arr[k] = right[j];
        j++;
        k++;
    }

}

void mergesort(int* arr, int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
```

```
        mergesort(arr, low, mid);
        mergesort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

void main()
{
        int a[15000],n,i,j,ch,temp;
        clock_t start,end;
        while(1)
        {
                printf("\n1:For manual entry of N value and array elements");
                printf("\n2:To display time taken for sorting number of elements N in the
range 500 to 14500");
                printf("\n3:To exit");
                printf("\nEnter your choice:");
                scanf("%d", &ch);
                switch(ch)
                {
                        case 1: printf("\nEnter the number of elements: ");
                                scanf("%d",&n);
                                printf("\nEnter array elements: ");
                                for(i=0;i<n;i++)
                                {
                                        scanf("%d",&a[i]);
                                }
                                start=clock();
```

```
                        mergesort(a, 0, n-1);
                        end=clock();
                        printf("\nSorted array is: ");
                        for(i=0;i<n;i++)
                                printf("%d\t",a[i]);
                        printf("\n%d,%f",n, (((double)(end-
start))/CLOCKS_PER_SEC));
                        break;
                case 2: n=500;
                        while(n<=14500)
                        {
                                for(i=0;i<n;i++)
                                {
                                        a[i]=n-i;
                                }
                                start=clock();
                                mergesort(a, 0, n-1);
                                //Dummy loop to create delay
                                for(j=0;j<500000;j++){ temp=38/600;}
                                end=clock();
                                printf("\n%d,%f",n, (((double)(end-
start))/CLOCKS_PER_SEC));
                                n=n+1000;
                        }
                        break;
                case 3: exit(0);
        }
        getchar();
    }
}
```

Output:

	A	B
1	Size	Time
2	500	0.00092
3	1500	0.001194
4	2500	0.001419
5	3500	0.001553
6	4500	0.000375
7	5500	0.000428
8	6500	0.000471
9	7500	0.000528
10	8500	0.000591
11	9500	0.000635
12	10500	0.000717
13	11500	0.000716
14	12500	0.000769
15	13500	0.000859
16	14500	0.000897



LeetCode Program related to sorting.

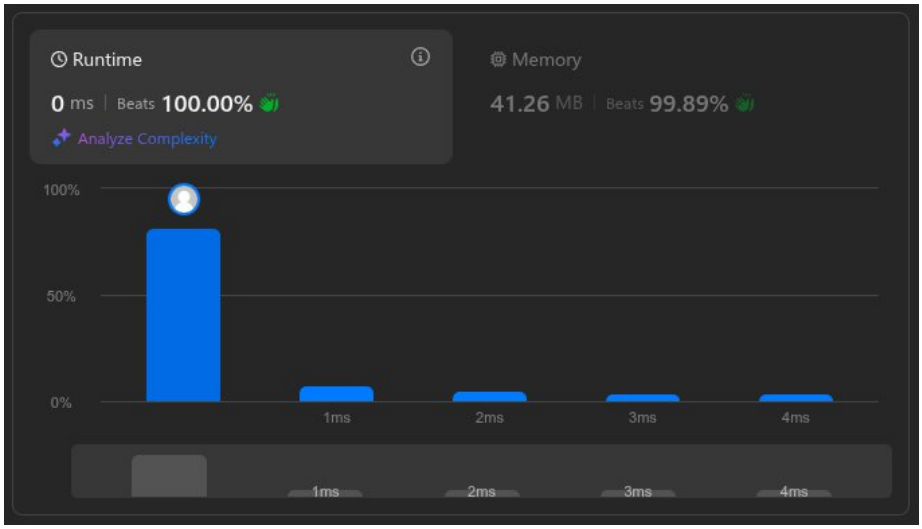
SORT COLORS

Code:

```
class Solution {
    public void sortColors(int[] nums) {
        int count0=0,count1=0,count2=0;
        for(int num:nums){
            switch(num){
                case 0 -> count0++;
                case 1 -> count1++;
                case 2 -> count2++;
            }
        }
        int ptr=0;
        while(ptr<nums.length){
            if(count0!=0){
                nums[ptr]=0;
                count0--;
            }
            else if(count1!=0){
                nums[ptr]=1;
                count1--;
            }
            else
            {
                nums[ptr]=2;
            }
            ptr++;
        }
```

```
    }
}
```

Output:


Runtime
0 ms | Beats 100.00%
Analyze Complexity
Memory
41.26 MB | Beats 99.89%
100%
50%
0%
1ms
2ms
3ms
4ms
1ms
2ms
3ms
4ms

Lab Program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>


void swap(int* a, int* b)

{

        int temp = *a;

        *a = *b;

        *b = temp;

}


int partition(int* arr, int low, int high)

{

        int i=low-1, j=low;

        int x = arr[high];

        for(;j < high;j++)

        {

                if(arr[j]<x)

                {

                        i++;

                        swap(&arr[j],&arr[i]);

                }

        }

        i++;
```

```
        swap(&arr[i],&arr[high]);
        return i;
}


void quicksort(int* arr, int low, int high)
{
        if(low >= high)
                return;
        int mid = partition(arr, low, high);
        quicksort(arr, low, mid-1);
        quicksort(arr, mid+1, high);
}

void main()
{
        int n;
        printf("Enter n:");
        scanf("%d",&n);
        printf("Enter array elements:");
        int arr[n];
        for(int i=0;i<n;i++)
                scanf("%d",&arr[i]);
        quicksort(arr, 0, n-1);
        printf("Sorted array:\n");
        for(int i=0;i<n;i++)
                printf("%d\t",arr[i]);
        printf("\n");
```

}

Output:

```
> ./quicksort
Enter n:5
Enter array elements:5 4 3 2 1
Sorted array:
1       2       3       4       5
```

Lab Program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>

#include <stdlib.h>


int left(int i){

        return 2*i;

}


int right(int i){

        return 2*i+1;

}


void maxHeapify(int* arr, int i, int size)

{

        int largest;

        int l = left(i);

        int r = right(i);

        if(l<size && arr[l]>arr[i])

                largest = l;

        else

                largest = i;

        if(r<size && arr[r]>arr[largest])

                largest = r;


        if(largest != i)
```

```
        {
                int temp = arr[i];
                arr[i] = arr[largest];
                arr[largest] = temp;
                maxHeapify(arr, largest, size);
        }
}

void build(int* arr, int n)
{
        for(int i=n/2;i>=0;i--)
                maxHeapify(arr, i, n);
}

void heapsort(int* arr, int n)
{
        build(arr, n);
        int size = n;
        for(int i=n-1;i>=1;i--)
        {
                int temp = arr[0];
                arr[0] = arr[i];
                arr[i] = temp;
                size--;
                maxHeapify(arr, 0, size);
        }
}
```

```
int main()
{
        int n;
        printf("Enter n:");
        scanf("%d",&n);
        int* a = (int*)malloc(n*sizeof(int));
        printf("Enter elements:");
        for(int i=0;i<n;i++)
                scanf("%d",&a[i]);
        heapsort(a, n);
        printf("Heap:\n");
        for(int i=0;i<n;i++)
                printf("%d ",a[i]);
        printf("\n");
}
```

Output:

```
> ./heapsort
Enter n:6
Enter elements:4 10 3 5 1 2
Heap:
1 2 3 4 5 10
```

Lab Program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include<stdio.h>
int i,j,n,c,w[10],p[10],v[10][10];
void knapsack(int n,int w[10],int p[10],int c)
{
	int max(int,int);
	for(i=0;i<=n;i++)
	{
		for(j=0;j<=c;j++)
		{
			if(i==0||j==0)
				v[i][j]=0;
			else if(w[i]>j)
				v[i][j]=v[i-1][j];
			else
				v[i][j]=max(v[i-1][j],(v[i-1][j-w[i]]+p[i]));

		}
	}
	printf("\n\n Maximum Profit is : %d ",v[n][c]);
	printf("\n\n\n Table : \n\n");
	for(i=0;i<=n;i++)
	{
		for(j=0;j<=c;j++)
		{
			printf("\t%d",v[i][j]);
```

```
                }
                printf("\n");
        }
}
int max(int a,int b)
{
        return ((a>b)?a:b);
}
void main()
{
        printf("\n Enter the no. of objects : ");
        scanf("%d",&n);
        printf("\n Enter the weights : ");
        for(i=1;i<=n;i++)
        {
                scanf("%d",&w[i]);
        }
        printf("\n Enter the Profits : ");
        for(i=1;i<=n;i++)
        {
                scanf("%d",&p[i]);
        }
        printf("\n Enter the capacity : ");
        scanf("%d",&c);
        knapsack(n,w,p,c);
}
```

Output:

```
> ./knapsack

Enter the no. of objects : 4

Enter the weights : 2 3 4 5

Enter the Profits : 3 4 5 6

Enter the capacity : 5


Maximum Profit is : 7


Table :

        0       0       0       0       0       0
        0       0       3       3       3       3
        0       0       3       4       4       7
        0       0       3       4       5       7
        0       0       3       4       5       7
```

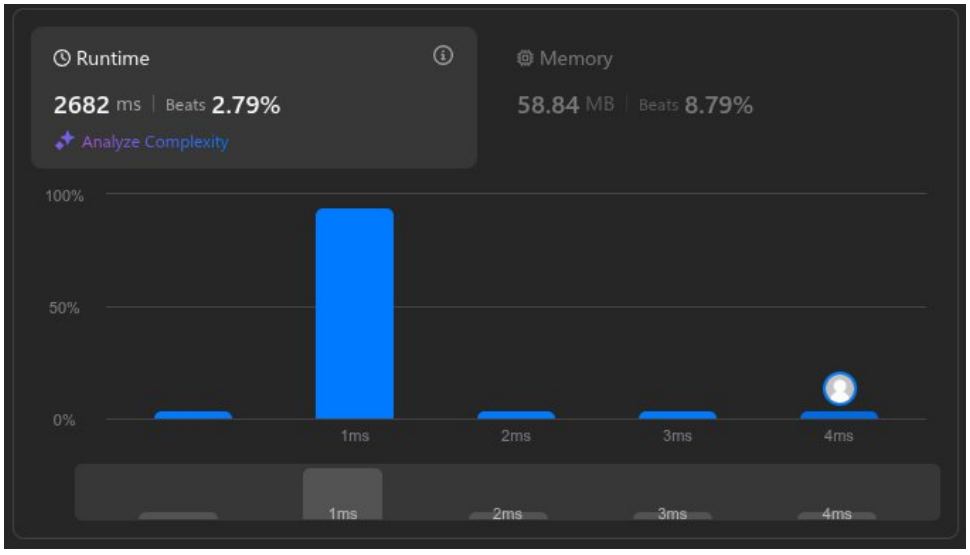
LeetCode Program related to Knapsack problem or Dynamic Programming.

MAXIMUM SUBARRAY

Code:

```
class Solution {
    public int maxSubArray(int[] nums) {
        int sum=0;
        int best= Integer.MIN_VALUE;
        for (int num: nums){
            System.out.print(num+"\t");
            sum=Math.max(num, sum+num);
            System.out.print(sum+"\t");
            best=Math.max(sum,best);
            System.out.println(best+"\t");
        }
        return best;
    }
}
```

Output:



Lab Program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

void floyd(int n, int edge[n][n])
{
        int p[n][n];
        for(int i=0;i<n;i++)
        {
                for(int j=0;j<n;j++)
                {
                        p[i][j]=edge[i][j];
                }
        }
        for(int k=0;k<n;k++)
        {
                for(int i=0;i<n;i++)
                {
                        for(int j=0;j<n;j++)
                        {
                                if(p[i][j]>p[i][k]+p[k][j])
                                        p[i][j]=p[i][k]+p[k][j];
                        }
                }
```

```
        }
        for(int i=0;i<n;i++)
        {
                for(int j=0;j<n;j++)
                {
                        printf("%d\t",p[i][j]);
                }
                printf("\n");
        }
}


void main()
{
        int n;
        printf("Enter n:");
        scanf("%d",&n);
        int edge[n][n];
        printf("Enter Adjacency Matrix:\n");
        for(int i=0;i<n;i++)
        {
                for(int j=0;j<n;j++)
                {
                        scanf("%d",&edge[i][j]);
                }
        }
        floyd(n, edge);
}
```

Output:

```
> ./floyd
Enter n:4
Enter Adjacency Matrix:
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0
0       5       8       9
999     0       3       4
999     999     0       1
999     999     999     0
```

LeetCode Program related to shortest distance calculation.

NETWORK DELAY TIME

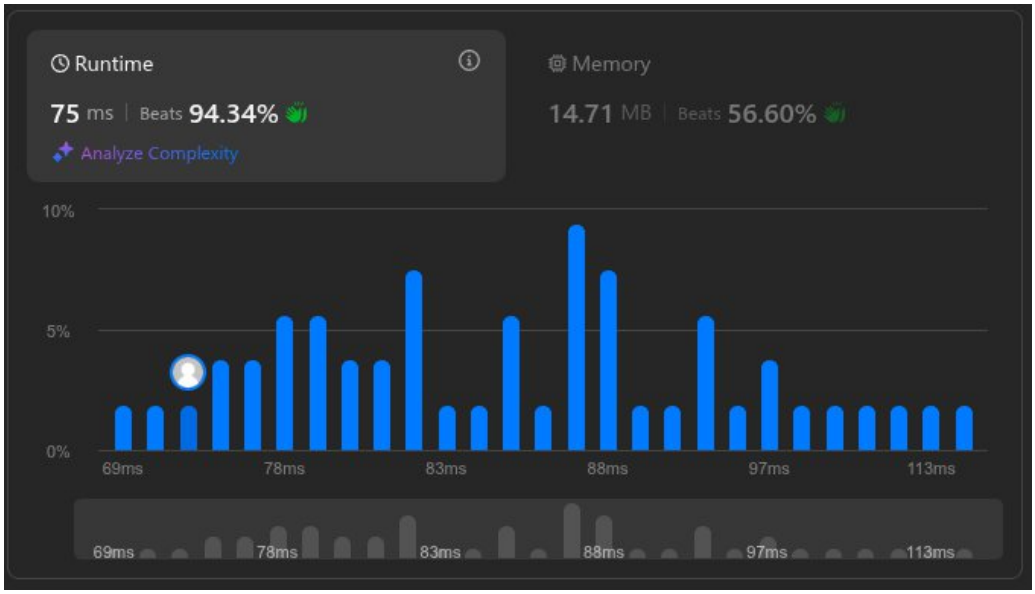
Code:

```
#define SOURCE times[i][0] - 1
#define DESTINATION times[i][1] - 1
#define WEIGHT times[i][2]
#define MAX 100001
#define MAXCOST 101

void init(int array[], int initializer, int n){
   for(int i = 0; i < n; i++){
      array[i] = initializer;
   }
}

int **prepareSparse(int n, int **times, int timeSize, int* timesColSize){
   int **matrix = (int **) malloc(sizeof(int *) * n);
   for(int i = 0; i < n; i++){
      matrix[i] = (int *) malloc(sizeof(int) * n);
      init(matrix[i], MAX, n);
   }

   for(int i = 0; i < timeSize; i++){
      matrix[SOURCE][DESTINATION] = WEIGHT;
   }
   return matrix;
}
```

```
int getMin(int cost[], int visited[], int n){
   int index = -1;
   int min = MAXCOST;
   for(int i = 0; i < n; i++){
      if(visited[i] == 0){
         if(index == -1 || cost[i] < min){
            min = cost[i];
            index = i;
         }
      }
   }
   return index;
}



int* SingleSourceShortestPath(int n, int **matrix, int k){
   int *cost = (int *) malloc(sizeof(int) * n);
   int *visited = (int *) malloc(sizeof(int) * n);
   int vertex;

   for(int i = 0; i < n; i++){
      cost[i] = matrix[k - 1][i];
      visited[i] = 0;
   }
   cost[k - 1] = 0;
   visited[k - 1] = 1;

   while((vertex = getMin(cost, visited, n)) != -1){
```

```
        for(int i = 0; i < n; i++){
            if(visited[i] == 0 && cost[i] > cost[vertex] + matrix[vertex][i]){
                cost[i] = cost[vertex] + matrix[vertex][i];
            }
        }
        visited[vertex] = 1;
    }

    return cost;
}

int networkDelayTime(int** times, int timesSize, int* timesColSize, int n, int k){
    int **matrix = prepareSparse(n, times, timesSize, timesColSize);
    int *cost = SingleSourceShortestPath(n, matrix, k);
    int max = -1;

    for(int i = 0; i < n; i++){
        max = (max > cost[i])?max:cost[i];
    }

    return (max < MAX)?max:-1;
}
```

Output:


Runtime
75 ms | Beats 94.34%
Analyze Complexity
Memory
14.71 MB | Beats 56.60%
10%
5%
0%
69ms
78ms
83ms
88ms
97ms
113ms

Lab Program 8:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

Code:

```
#include <stdio.h>

#include <stdlib.h>


#define MAX 10

#define INF 999


typedef struct {

   int u, v, weight;

} Edge;


int parent[MAX], rank[MAX];

Edge edges[MAX * MAX];

int n, m;


void initSet() {

   for (int i = 0; i < n; i++) {

      parent[i] = i;

      rank[i] = 0;

   }

}


int find(int i) {
```

```
    if (i != parent[i]) {
        parent[i] = find(parent[i]);
    }
    return parent[i];
}

void unionSet(int i, int j) {
    int root1 = find(i);
    int root2 = find(j);

    if (root1 != root2) {
        if (rank[root1] > rank[root2]) {
            parent[root2] = root1;
        } else if (rank[root1] < rank[root2]) {
            parent[root1] = root2;
        } else {
            parent[root2] = root1;
            rank[root1]++;
        }
    }
}

int compare(const void *a, const void *b) {
    return ((Edge *)a)->weight - ((Edge *)b)->weight;
}

void kruskal() {
    int mstWeight = 0;
```

```
    int edgeCount = 0;

    qsort(edges, m, sizeof(Edge), compare);

    for (int i = 0; i < m; i++) {
        int u = edges[i].u;
        int v = edges[i].v;

        if (find(u) != find(v)) {
            printf("Edge (%d, %d) with weight %d\n", u, v, edges[i].weight);
            unionSet(u, v);
            mstWeight += edges[i].weight;
            edgeCount++;

            if (edgeCount == n - 1) {
                break;
            }
        }
    }

    printf("\nTotal weight of MST: %d\n", mstWeight);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &m);
```

```
    printf("Enter the edges (u, v, weight):\n");
    for (int i = 0; i < m; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    initSet();
    kruskal();

    return 0;
}
```

Output:

```
> ./kruskal
Enter number of vertices: 6
Enter number of edges: 10
Enter the edges (u, v, weight):
0 1 3
1 2 1
1 5 4
2 5 4
2 3 6
0 5 5
0 4 6
3 5 5
3 4 8
4 5 2
Edge (1, 2) with weight 1
Edge (4, 5) with weight 2
Edge (0, 1) with weight 3
Edge (1, 5) with weight 4
Edge (3, 5) with weight 5

Total weight of MST: 15
```

Code:

```
#include <stdio.h>
```

```
int cost[10][10], n, t[10][2], sum = 0;

void prims() {
   int i, j, u, v;
   int min, source;
   int p[10], d[10], s[10];

   source = 0;
   min = 999;

   for (i = 0; i < n; i++) {
      d[i] = (cost[source][i] == 0 && i != source) ? 999 : cost[source][i];
      s[i] = 0;
      p[i] = source;
   }

   s[source] = 1;
   sum = 0;

   int k = 0;
   for (i = 0; i < n - 1; i++) {
      min = 999;
      u = -1;

      for (j = 0; j < n; j++) {
         if (s[j] == 0 && d[j] < min) {
            min = d[j];
            u = j;
```

```
        }
      }

      if (u != -1) {
        t[k][0] = u;
        t[k][1] = p[u];
        k++;

        sum += cost[u][p[u]];

        s[u] = 1;

        for (v = 0; v < n; v++) {
          if (s[v] == 0 && cost[u][v] != 0 && cost[u][v] < d[v]) {
            d[v] = cost[u][v];
            p[v] = u;
          }
        }
      }
    }
}

void main() {
  int i, j;

  printf("Enter n: ");
  scanf("%d", &n);

  printf("Enter the cost adjacency matrix:\n");
```

```
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    prims();

    printf("Edges of MST:\n");
    for (i = 0; i < n - 1; i++) {
        printf("(%d, %d)\n", t[i][0], t[i][1]);
    }

    printf("\nMinimal Sum: %d\n", sum);
}
```

Output:

```
> ./prims
Enter n: 6
Enter the cost adjacency matrix:
0 3 0 0 6 5
3 0 1 0 0 4
0 1 0 6 0 4
0 0 6 0 8 5
6 0 0 8 0 2
5 4 4 5 2 0
Edges of MST:
(1, 0)
(2, 1)
(5, 1)
(4, 5)
(3, 5)

Minimal Sum: 15
```

Lab Program 9:

Implement Fractional Knapsack using Greedy technique.

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct obj {
   int v;
   int w;
} obj;

int compare(const void *a, const void *b) {
   obj *objA = (obj *)a;
   obj *objB = (obj *)b;
   float ratioA = (float)objA->v / objA->w;
   float ratioB = (float)objB->v / objB->w;
   if (ratioA > ratioB) {
      return -1;
   } else if (ratioA < ratioB) {
      return 1;
   } else {
      return 0;
   }
}

void fracKnap(obj *arr, int n, int max) {
   qsort(arr, n, sizeof(obj), compare);
   float *qty = (float *)malloc(n * sizeof(float));
```

```
    for (int i = 0; i < n; i++) {
        qty[i] = 0.0;
    }

    int i = 0;
    for (; i < n; i++) {
        if (arr[i].w <= max) {
            qty[i] = 1.0;
            max -= arr[i].w;
        } else {
            break;
        }
    }

    if (i < n) {
        qty[i] = (float)max / arr[i].w;
        max = 0;
    }

    float val = 0;
    for (int i = 0; i < n; i++) {
        val += qty[i] * arr[i].v;
        printf("Quantity of item %d: %f\n",arr[i].v, qty[i]);
    }

    printf("\nTotal value of knapsack: %f\n", val);
    free(qty);
}
```

```
int main() {
   int n;
   printf("Enter the number of items: ");
   scanf("%d", &n);

   obj *arr = (obj *)malloc(n * sizeof(obj));

   printf("Enter value and weight pairs:\n");
   for (int i = 0; i < n; i++) {
      scanf("%d %d", &arr[i].v, &arr[i].w);
   }

   int max;
   printf("Enter the maximum weight of the knapsack: ");
   scanf("%d", &max);

   fracKnap(arr, n, max);
   free(arr);

   return 0;
}
```

Output:

```
master@webserve:/mnt/DRIVE/SHASHANK/BMSCE/IV SEM/lab/Lab-6b-04-30-25$ ./fracKnapsack
Enter the number of items: 5
Enter value and weight pairs:
10 20
20 10
60 30
40 30
50 15
Enter the maximum weight of the knapsack: 100
Quantity of item 50: 1.000000
Quantity of item 20: 1.000000
Quantity of item 60: 1.000000
Quantity of item 40: 1.000000
Quantity of item 10: 0.750000

Total value of knapsack: 177.500000
```

LeetCode Program related to Greedy Technique algorithms.

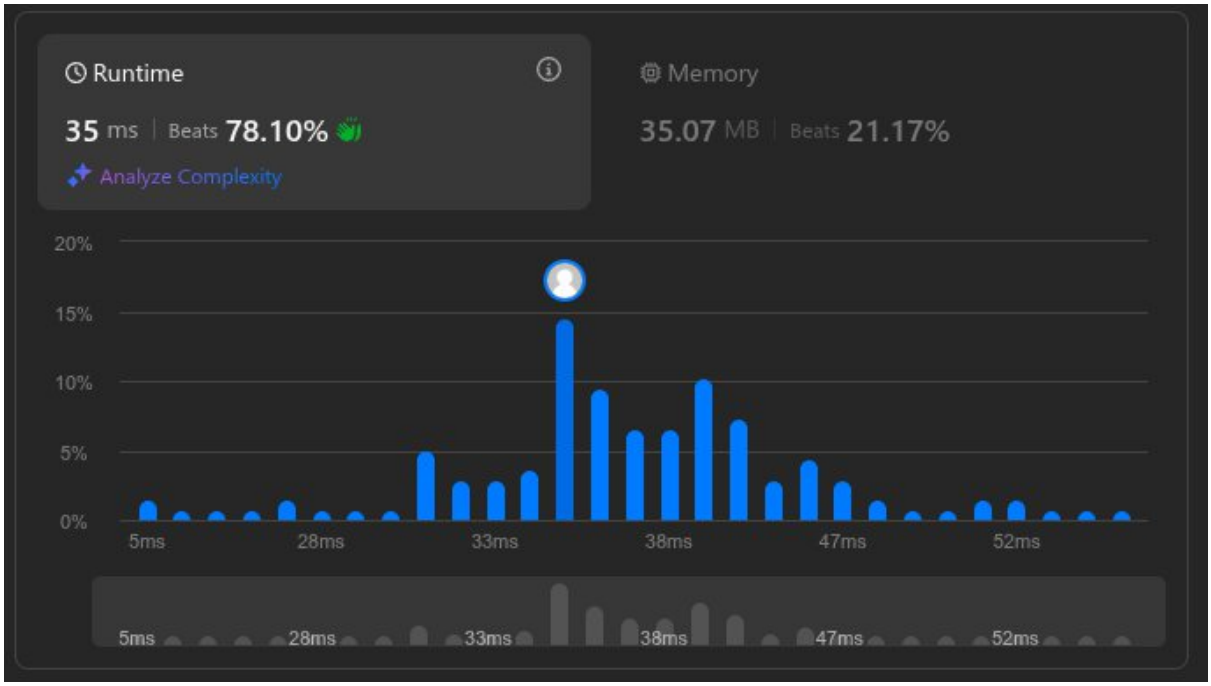
NON OVERLAPPING INTERVALS

Code:

```
int compare(const void *a,const void *b)
{
   int *intervals_A=*(int**)a;
   int *intervals_B=*(int**)b;
   return intervals_A[0]-intervals_B[0];
}



int eraseOverlapIntervals(int** intervals, int intervalsSize, int* intervalsColSize)
{
   int count=0;

 qsort(intervals,intervalsSize,sizeof(int*),compare);

int prev_End=intervals[0][1];
for(int i=1;i<intervalsSize;i++)

if(intervals[i][0]<prev_End)
{
   count++;
   prev_End=(intervals[i][1]<prev_End)?intervals[i][1]:prev_End;
}
   else
   {
```

```
        prev_End=intervals[i][1];
    }

return count;

}
```

Output:



Lab Program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

Code:

```
#include<stdio.h>

void main()

{

        int i,j,n,v,k,min,u,c[20][20],s[20],d[20];


        printf("\n Enter the no. of vertices : ");

        scanf("%d",&n);

        printf("\n Enter the cost adjacency matrix : ");

        printf("\n Enter 999 for no edge : \n");

        for(i=1;i<=n;i++)

        {

                for(j=1;j<=n;j++)

                {

                        scanf("%d",&c[i][j]);

                }

        }

        printf("\n Enter the source vertex : ");

        scanf("%d",&v);

        for(i=1;i<=n;i++)

        {

                s[i]=0;

                d[i]=c[v][i];

        }

        d[v]=0;
```

```
s[v]=1;
for(k=2;k<=n;k++)
{
        min=999;
        for(i=1;i<=n;i++)
        {
                if((s[i]==0)&&(d[i]<min))
                {
                        min=d[i];
                        u=i;
                }
        }
        s[u]=1;
        for(i=1;i<=n;i++)
        {

                if(s[i]==0)
                {
                        if(d[i]>(d[u]+c[u][i]))
                        {
                                d[i]=d[u]+c[u][i];
                        }
                }

        }
}
printf("\n The shortest distance from %d is ",v);
for(i=1;i<=n;i++)
```

```
        {
                printf("\n %d --> %d = %d ",v,i,d[i]);
        }
        printf("\n");
}
```

Output:

```
> ./dijkstra

 Enter the no. of vertices : 5

 Enter the cost adjacency matrix :
 Enter 999 for no edge :
0 10 999 30 100
999 0 50 999 999
999 999 0 999 10
999 999 20 0 60
999 999 999 999 0

 Enter the source vertex : 1

 The shortest distance from 1 is
 1 --> 1 = 0
 1 --> 2 = 10
 1 --> 3 = 50
 1 --> 4 = 30
 1 --> 5 = 60
```

Lab Program 11:

Implement "N-Queens Problem" using Backtracking.

Code:

```
#include <stdio.h>
#include <stdbool.h>

#define N 4

bool isSafe(int board[], int row, int col) {
   for (int i = 0; i < row; i++) {
      if (board[i] == col ||
         board[i] - i == col - row ||
         board[i] + i == col + row) {
         return false;
      }
   }
   return true;
}

void printSolution(int board[]) {
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         if (board[i] == j) {
            printf("Q ");
         } else {
            printf(". ");
         }
      }
```

```
        printf("\n");
    }
    printf("\n");
}

bool solveNQueens(int board[], int row) {
    if (row == N) {
        printSolution(board);
        return true;
    }

    bool res = false;
    for (int col = 0; col < N; col++) {
        if (isSafe(board, row, col)) {
            board[row] = col;
            res = solveNQueens(board, row + 1) || res;
            board[row] = -1;
        }
    }
    return res;
}

int main() {
    int board[N];
    for (int i = 0; i < N; i++) {
        board[i] = -1;
    }
```

```
    if (!solveNQueens(board, 0)) {
        printf("No solution exists.\n");
    }

    return 0;
}
```

Output: